Adapting Audio-JEPA for Low-Resource Thai Automatic Speech Recognition

Sawit Koseeyaumporn, Kittipong Tapyou, Warapol Pratumta
Computer Engineering
King Mongkut's University of Technology Thonburi
Bangkok, Thailand

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction

Automatic speech recognition (ASR) converts speech into text, but supervised systems typically depend on large amounts of transcribed audio. Accurate transcripts require time and human effort, so this requirement is difficult to meet when labeled speech is scarce [1]. Thai is such a case. Paired speech–text data remains limited compared with high-resource languages [2], and the language brings additional difficulties: it is tonal, distinguishes short and long vowels, and is written without spaces between words. In this work, "low-resource" refers primarily to a limited transcription budget. The availability of unlabeled audio and of computation is reported separately.

Conventional ASR describes the speech signal with fixed acoustic features such as mel-frequency cepstral coefficients (MFCCs) or Mel-spectrograms [3]. Self-supervised learning (SSL) instead learns representations directly from unlabeled audio, which is far cheaper to collect than transcripts. Models such as wav2vec 2.0 and HuBERT pretrain an encoder on untranscribed speech and then adapt it to recognition with few labeled examples [4], [5]. The question therefore shifts from which features to compute to how an encoder should learn its representations from unlabeled audio.

This paper follows a representation-based approach. Instead of reconstructing the input signal or predicting discrete pseudo-labels, the encoder learns by predicting the *representations* of hidden parts of the input from the visible context. Joint-embedding predictive architectures (JEPAs) implement this principle [6], [7], and Audio-JEPA applies it to audio by predicting latent features of masked Mel-spectrogram patches [8]. Audio-JEPA learns competitive general-purpose audio representations with a modest training budget. However, it was pretrained on general audio rather than speech, evaluated mainly on classification tasks, and never applied to continuous transcription or to Thai. Its coarse patch grid also produces one time step every 625 ms, which is too coarse for character-level CTC alignment. Whether JEPA-style representation prediction can reduce the labeled-data requirement of Thai ASR is therefore an open question.

We investigate this question with a two-stage framework. First, a speech-adapted Audio-JEPA is pretrained, or further pretrained, on unlabeled Thai speech. Second, its encoder is transferred to a recognizer with a connectionist temporal classification (CTC) head [9] and fine-tuned under controlled labeled-data budgets. The contributions of this work are as follows:

- We adapt Audio-JEPA to speech by replacing its 16×16 spectrogram patches with a full-frequency projection stem that produces one token every 20 ms, and by replacing two-dimensional block masking with short temporal-span masking. All Transformer blocks of the original model can still be transferred.
- We define a JEPA pretraining and CTC fine-tuning pipeline for Thai, including valid-length handling, CTC feasibility checks, and representation-collapse monitoring.
- We design a controlled comparison that separates the contribution of upstream Audio-JEPA weights, in-domain Thai JEPA pretraining, and supervised training alone, evaluated with 1, 10, and 50 hours and the full set of labeled Thai speech using character and word error rates.

The rest of the paper is organized as follows. Section II reviews related work. Section III describes the proposed method and experimental design. Section IV will report the experimental results, and Section V will conclude the paper.

II. Related Work

A. Evolution of Speech Representation Learning

Speech representation learning has evolved from fixed acoustic features, through supervised neural models, to models pretrained on unlabeled audio [4], [10], [11].

1) Handcrafted and statistical models: Early speech recognition systems divided audio into short frames and extracted features with fixed mathematical transformations [3], [10], [12], [13]. MFCCs compress the short-time spectrum according to properties of human hearing, producing compact representations [3]. Perceptual linear prediction (PLP) incorporates further auditory characteristics, improving robustness across speakers and recording conditions [12]. Temporal dynamics were captured by appending differences between adjacent

frames. These features were passed to statistical models: hidden Markov models (HMMs) represented the timing and sequence of speech [13], and Gaussian mixture models (GMMs) estimated acoustic likelihoods [10]. Although computationally efficient, this pipeline separated feature extraction, acoustic modeling, and language modeling, which prevented joint optimization.

2) Supervised methods: Supervised neural methods learn directly from transcribed speech. Deep neural networks replaced the GMM acoustic model and substantially improved accuracy [11]. Convolutional neural networks captured local spectro-temporal patterns [14], and recurrent neural networks modeled long-range temporal context [15]. Connectionist temporal classification (CTC) made it possible to train sequence models without frame-level alignments by summing over all alignments consistent with the transcript [9]. Attention-based encoder–decoder models such as Listen, Attend and Spell map audio directly to characters [16], and the Conformer combines self-attention and convolution to model both global context and local detail [17]. These models simplified the pipeline and improved accuracy, but they still require large quantities of labeled audio.

3) Self-supervised methods: Self-supervised methods learn from audio without transcripts by deriving their training targets from the signal itself. Contrastive predictive coding (CPC) identifies the true future representation among negative candidates [18], while autoregressive predictive coding (APC) predicts future acoustic frames directly [19]. Later models, including wav2vec 2.0 [4], HuBERT [5], WavLM [20], and data2vec [21], mask parts of the input and learn from bidirectional context. Pretrained on unlabeled audio, these models can be adapted to downstream tasks with substantially less labeled data [22].

B. Self-Supervised Learning Paradigms

Self-supervised speech methods can be grouped into four paradigms according to what they predict [22].

1) Reconstruction: Reconstruction methods hide or corrupt parts of the input and train the model to restore them. Mockingjay masks acoustic frames and reconstructs them with a Transformer encoder [23]. DeCoAR reconstructs frames from bidirectional context [24], and TERA alters the input along the time, frequency, and magnitude axes [25]. The objective is simple and requires neither negative samples nor clustering. However, the model must also spend capacity on background noise and low-level acoustic details that are irrelevant to recognition.

2) Contrastive learning: Contrastive methods train the model to pick out the correct latent target from a set of distractors. CPC applies this idea to future prediction [18]. Wav2vec 2.0 masks the latent speech sequence and identifies the correct quantized representation for each masked position among distractors sampled from the same utterance [4]. Predicting high-level latent targets is more useful than reconstructing raw input, but performance depends on the negative-sampling strategy, and the quantization codebook adds training complexity.

3) Discrete unit prediction: Discrete-unit methods convert continuous speech into categorical targets and train the model to predict them at masked positions. HuBERT obtains its targets by clustering acoustic features and later its own representations [5]. WavLM extends masked unit prediction with simulated noise and overlapping speech to improve robustness [20]. Categorical targets suit speech recognition well and avoid explicit negatives, but the learned representations depend on the quality of the offline clustering, and noisy pseudo-labels can limit them.

4) Representation prediction: Representation-prediction methods regress continuous feature vectors rather than reconstructing the input or predicting discrete labels. In data2vec, a student network predicts contextualized representations produced by an EMA teacher that observes the complete input [21]. JEPAs [6], [7] use a context encoder and a predictor to estimate target-encoder representations of masked regions. This paradigm requires no input-reconstruction objective, no negative examples, and no externally generated clusters. It is the paradigm we adopt in this work.

C. Joint-Embedding Predictive Architectures

1) From images to audio: Self-supervised image methods have mainly followed two families [7]. Invariance-based methods train an encoder to produce similar embeddings for multiple augmented views of the same input. They learn semantic features, but their hand-crafted augmentations introduce strong biases that may not transfer to other data distributions. Generative methods mask portions of the input and reconstruct the missing content, which requires less prior knowledge but tends to produce lower-level representations. I-JEPA combines the strengths of the two: from a visible context block, a predictor estimates the *embeddings* of target blocks produced by an EMA target encoder [7]. Prediction in representation space encourages semantic features without the burden of reconstructing every pixel and without augmentation-specific priors. For speech, this suggests an encoder that captures phonetic and linguistic structure without having to reconstruct the waveform or spectrogram exactly.

A-JEPA transferred this principle to audio spectrograms, using a curriculum that moves from random block masking to time–frequency-aware masking, and reported strong results on audio and speech classification benchmarks [26]. Riou et al. showed that the choice of context and target regions strongly affects audio representation quality, and that design choices successful for images do not necessarily transfer to audio [27].

2) Audio-JEPA: Audio-JEPA treats a Mel-spectrogram as an image. It splits the spectrogram into 16×16 patches, randomly masks patch regions, and uses a Vision Transformer context encoder and predictor to regress the target encoder's representations of the masked patches [8]. Pretrained on approximately 5,300 hours of AudioSet, it achieved results comparable to wav2vec 2.0 and data2vec on several tasks of the X-ARES benchmark while using a smaller reported

training budget. Its results are mixed, however: Audio-JEPA is stronger on several general-audio and music tasks but weaker on tasks that require fine-grained speech discrimination, and its evaluation uses frozen representations for classification-style tasks rather than transcription.

3) Speech-specific JEPA: Recent preprints investigate JEPAs designed specifically for speech. GMM-Anchored JEPA uses frozen soft posteriors from a GMM fitted on log-Mel features as auxiliary targets. The authors report representation collapse without such grounding in their setting, and improved ASR, emotion recognition, and slot filling with roughly 50,000 hours of pretraining speech [28]. S-JEPA predicts soft GMM posteriors through a Kullback–Leibler objective and reports the lowest WER among its evaluated SSL models below 90 million parameters under the SUPERB protocol [29]. These studies show that JEPA-style learning can serve ASR, but they rely on large English pretraining corpora and additional clustering targets, and they do not study low-resource or tonal languages.

D. Self-Supervised Learning for Low-Resource ASR

1) Pretraining with limited transcripts: The standard workflow for low-resource ASR is to pretrain an encoder on a large amount of unlabeled audio, then fine-tune it with a small transcribed set and a recognition head such as CTC, a transducer, or an encoder–decoder [22]. Wav2vec 2.0 demonstrated this workflow at scale. With 53,000 hours of unlabeled audio, a Transformer language model, and only ten minutes of labeled data, it reached 4.8%/8.2% WER on LibriSpeech test-clean/test-other. With one hour of labels, it outperformed the previous state of the art trained on 100 hours [4]. HuBERT matched or improved on wav2vec 2.0 across fine-tuning sets from ten minutes to 960 hours [5]. These results show strong label efficiency under large-scale pretraining, although model size, pretraining data, and decoding differ across studies.

2) Multilingual transfer: Cross-lingual pretraining extends these benefits to languages with little data. XLSR pretrains a single wav2vec-style model on many languages and significantly outperforms monolingual pretraining. It reported a 72% relative phone error rate reduction on CommonVoice compared with the best previously known results and a 16% relative WER improvement on BABEL compared with a comparable system [1]. XLS-R scales this approach to 436,000 hours in 128 languages [30], and the Massively Multilingual Speech project covers more than 1,100 languages for ASR [31]. Large weakly supervised models such as Whisper are another route to multilingual ASR, but they rely on hundreds of thousands of hours of paired audio and text [32].

3) Thai ASR: For Thai, XLSR-Thai continues self-supervised pretraining on 36,000 hours of Thai speech [2], and Typhoon ASR Real-time trains a FastConformer-Transducer with systematic text normalization for numbers, loanwords, and repetition markers [33]. The latter work shows that consistent transcript normalization matters for both training and evaluating Thai ASR. Public Thai speech corpora, including the Porjai corpus [34], now make controlled studies of label efficiency possible. Berrebbi et al. also show that self-supervised features can suffer under domain mismatch in low-resource recognition [35], which motivates in-domain pretraining on Thai audio.

E. Research Gap

The literature establishes that self-supervised pretraining improves low-resource ASR, but this evidence comes almost entirely from contrastive and discrete-unit models such as wav2vec 2.0, HuBERT, WavLM, and XLS-R. Audio-JEPA provides efficient latent audio representation learning, yet it (i) was trained on general audio rather than speech, (ii) was evaluated on audio understanding and classification rather than transcription, (iii) uses a temporal resolution too coarse for CTC, and (iv) has not been studied for Thai. Speech-specific JEPAs address ASR but rely on large English corpora. To our knowledge, JEPA-style representation prediction has not been evaluated for low-resource Thai ASR. This work addresses that gap.

III. Methodology

A. Overview

We adopt the representation-based approach introduced in Section I: an encoder first learns speech representations from unlabeled Thai audio, and the same encoder is then adapted to transcription using a limited amount of labeled speech. Fig. 1 summarizes the two stages. In Stage 1 (pretraining), log-Mel features computed from untranscribed Thai recordings are passed to a speech-adapted Audio-JEPA, which learns by predicting the latent features of hidden time spans [8]. In Stage 2 (fine-tuning), the pretrained context encoder is transferred to a recognizer, a connectionist temporal classification (CTC) head [9] is attached, and the model is trained on Thai transcripts under fixed labeled-data budgets. The JEPA predictor and target encoder are discarded after Stage 1, so they add no inference cost.

B. Dataset and Text Processing

Experiments use the Porjai Central Thai speech corpus released by CMKL [34]. Because the corpus does not provide speaker identifiers, utterances are assigned to training, validation, and test partitions (80%/10%/10%) by a deterministic hash of the utterance identifier. Every run therefore uses identical partitions. A consequence is that the partitions are utterance-disjoint but not guaranteed to be speaker-disjoint; we treat this as a limitation when interpreting test results. Self-supervised pretraining reads audio only from the training partition. Validation and test audio is never seen during Stage 1, and transcripts are never seen in Stage 1 at all.

Audio is decoded, converted to mono, and resampled to 16 kHz. Utterances outside the permitted duration range are removed from training instead of being truncated, so every training waveform remains consistent with its full transcript. The number of excluded utterances and hours is logged for each run.

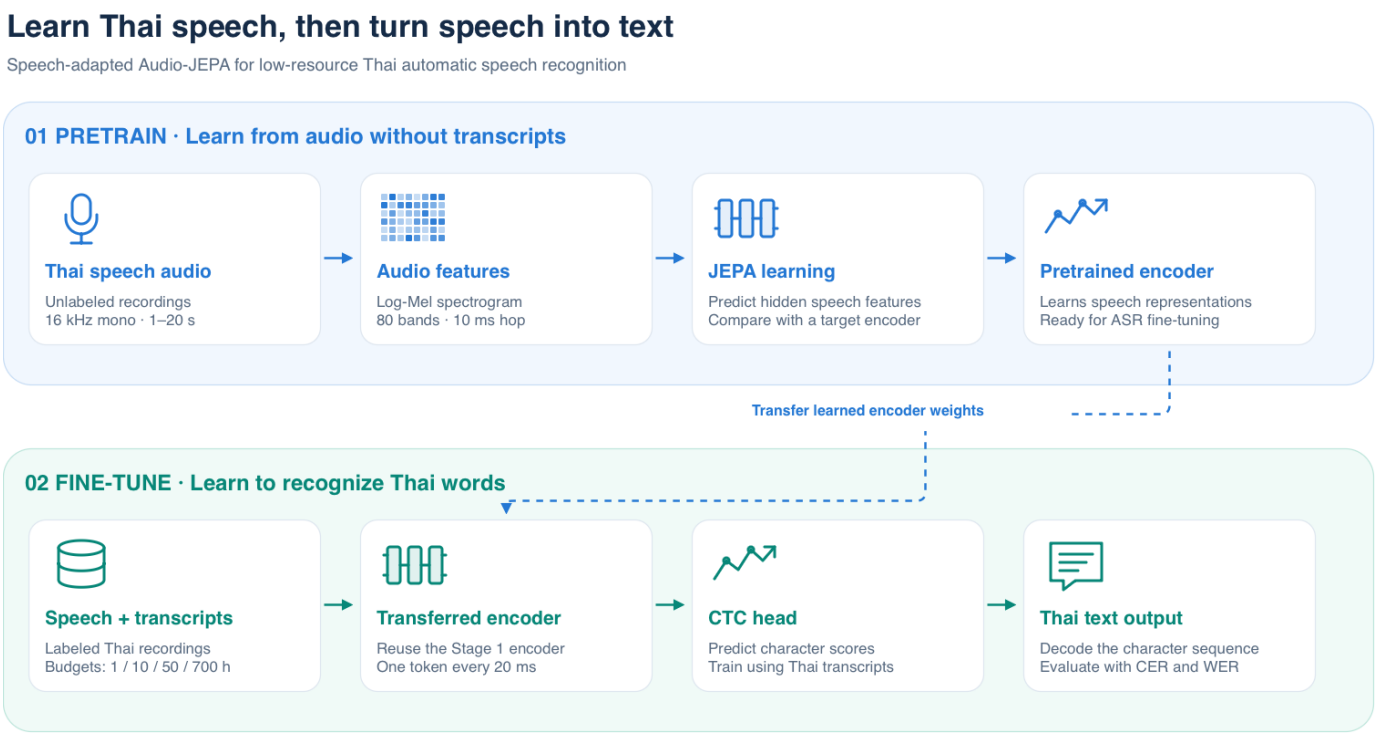


Fig. 1. Overall framework. Stage 1 pretrains a speech-adapted Audio-JEPA encoder on unlabeled Thai audio. Stage 2 transfers the learned encoder weights to a CTC recognizer trained with 1, 10, 50, or approximately 700 hours of transcribed Thai speech and evaluated with CER and WER.

Transcripts are normalized with a single rule set that is applied identically to training labels, references, and hypotheses: Unicode NFKC normalization, removal of zero-width characters and punctuation, mapping of Thai digits to Arabic digits, lowercasing of Latin letters, and collapsing of repeated whitespace. Thai vowels, tone marks, and the repetition mark are retained. The output vocabulary is character-level and includes Thai consonants, vowels, tone and orthographic marks, digits, Latin letters, a word-boundary space token, an unknown token, and the CTC blank at index 0. Characters outside the vocabulary map to the unknown token during training, but references keep their original characters, so such characters still count as errors at evaluation.

C. Acoustic Frontend and Speech Tokenization

Each waveform is converted to 80 log-Mel filterbank features using a 25 ms analysis window and a 10 ms hop. The original Audio-JEPA divides a 256×128 spectrogram into 16×16 patches, which yields only 16 temporal positions for a 10-second clip, or one position every 625 ms [8]. CTC requires that the number of output frames T satisfies

$$T \geq U + R, \tag{1}$$

where U is the transcript length and R is the number of adjacent repeated labels, because repeated labels must be separated by a blank. A character-level Thai transcript typically has more characters than the coarse grid has time steps, so most utterances cannot be aligned at that resolution.

We therefore replace the square patch embedding with a full-frequency temporal projection stem. A two-dimensional convolution with kernel and stride $(80, 2)$ and 768 output channels maps each pair of consecutive 10 ms frames, across all 80 Mel bands, to one 768-dimensional token. The encoder thus receives one token every 20 ms (50 tokens per second), as shown in Fig. 2. The exact number of valid tokens is computed from the unpadded waveform length:

$$F = \left\lfloor \frac{N - W}{H} \right\rfloor + 1, \qquad T = \left\lfloor \frac{F - 2}{2} \right\rfloor + 1, \tag{2}$$

where N is the number of samples, W and H are the window and hop lengths in samples, F is the number of filterbank frames, and T is the number of encoder tokens. These valid lengths, rather than padded tensor lengths, define the attention masks, the masking region in Stage 1, and the CTC input lengths in Stage 2. Before training, every batch is checked against (1), and the ratio $T/(U + R)$ is reported.

D. Encoder Architecture

The encoder keeps the Audio-JEPA Vision Transformer backbone: 12 pre-LayerNorm Transformer blocks with width 768, 12 attention heads, and a feed-forward width of 3072 with GELU activations, followed by a final LayerNorm [8]. Fixed sinusoidal positional embeddings are added to the tokens along the time axis. Padded positions are excluded from self-attention by an additive attention mask. The encoder has no classification token and performs no temporal pooling, so its output is an ordered sequence of 20 ms frame representations.

The model attends bidirectionally and targets offline transcription.

E. Stage 1: Thai Speech JEPA Pretraining

Fig. 2 details the pretraining objective. Let P_θ denote the projection stem and E_θ the context encoder. A target copy $(P_{\bar{\theta}}, E_{\bar{\theta}})$ is initialized with identical weights and receives no gradients.

1) Short-span temporal masking: The original Audio-JEPA samples random two-dimensional blocks on the spectrogram patch grid [8]. Speech information is organized mainly along time, so we mask whole time steps across all frequencies. For each batch, spans of five consecutive tokens (100 ms, roughly phone-to-syllable scale) are sampled within the valid region until approximately 50% of the valid tokens are hidden. The context branch receives only the remaining visible tokens, each with its original positional embedding. Prediction targets are drawn from the hidden spans, with two target sets per batch. Padding is never selected as context or target. Hiding short spans prevents the model from solving the task by copying immediately adjacent frames, while leaving enough surrounding context to infer the hidden content.

2) Predictor: The predictor is a narrower Transformer with 6 blocks of width 384 and 12 heads. Context outputs are linearly projected to width 384. A shared learnable mask token, plus the positional embedding of each hidden position, is appended for every target position, and the sequence is processed by the predictor blocks. A final linear layer maps the outputs back to 768 dimensions, giving the prediction $\hat{z}_t$ for each target position t.

3) Latent prediction objective: Let z_t be the target encoder output at position t, and let $\bar{z}_t$ be z_t standardized across its channels. Following the normalized latent loss of Audio-JEPA, we minimize the squared distance between ℓ_2-normalized prediction and target vectors over the set of target positions M:

$$\begin{aligned}\mathcal{L}_{\mathrm{JEPA}} &= \frac{1}{|M|}\sum_{t\in M}\left\|\frac{\hat{z}_t}{\|\hat{z}_t\|_2} - \mathrm{sg}\left[\frac{\bar{z}_t}{\|\bar{z}_t\|_2}\right]\right\|_2^2 \\ &= \frac{1}{|M|}\sum_{t\in M}\big(2 - 2\cos(\hat{z}_t, \bar{z}_t)\big),\end{aligned} \tag{3}$$

where $\mathrm{sg}[\cdot]$ stops gradients. The loss is computed only at hidden positions and is averaged over the target sets. Because the targets are features rather than spectrogram values, the model is not required to reproduce low-level acoustic detail such as background noise.

4) EMA target update: Only the context stem, context encoder, and predictor are optimized. After each update, the target parameters follow an exponential moving average of the context parameters:

$$\bar{\theta} \leftarrow \tau_s\bar{\theta} + (1-\tau_s)\theta, \qquad \tau_s = 1 - (1-\tau_0)\,\frac{1+\cos(\pi s/S)}{2}, \tag{4}$$

with $\tau_0 = 0.996$, so τ_s rises from 0.996 to 1.0 over the S pretraining updates. Dropout is disabled in the target branch.

5) Collapse monitoring: Latent prediction objectives admit a trivial solution in which all embeddings become constant. The EMA target and stop-gradient discourage this solution but do not rule it out. During pretraining we therefore log the per-channel variance of target features, the mean cosine similarity between utterance-level embeddings within a batch, and the mean feature norm. A run whose loss falls while feature variance approaches zero or inter-utterance similarity approaches one is treated as collapsed and is not used for fine-tuning.

6) Initialization: Stage 1 is run from two initializations. In the transfer setting, all Transformer blocks and the final LayerNorm are loaded from the public Audio-JEPA checkpoint pretrained on AudioSet [8], [36]. The projection stem cannot be loaded because its kernel shape differs from the original 16×16 patch embedding, so it is initialized randomly. The pretraining then continues on Thai audio. In the scratch setting, all weights are initialized randomly. A loading report records every loaded, reinitialized, and skipped tensor together with the fraction of parameters transferred.

F. Stage 2: CTC Fine-Tuning

After pretraining, the context stem and context encoder are kept. The predictor and target branch are removed. A dropout layer ($p = 0.1$) and a linear layer map each 768-dimensional frame representation to a distribution over the vocabulary $\mathcal{V}$ and the blank symbol $\varnothing$. For an input x with transcript y, training minimizes the CTC loss

$$\mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\pi\in\mathcal{B}^{-1}(y)}\prod_{t=1}^{T} p(\pi_t \mid x), \tag{5}$$

where $\mathcal{B}$ merges consecutive identical labels and then removes blanks [9]. Input lengths are the valid token counts defined in Section III-C. Infinite losses are not replaced with zero, so any alignment failure is visible in the logs.

Because the projection stem has been retrained or newly initialized, the optimizer uses three parameter groups with separate learning rates: the Transformer backbone, the projection stem, and the CTC head. Pretrained backbones use a learning rate one-tenth of the head learning rate, whereas the stem and head use larger rates. When a frozen-encoder variant is run, only the Transformer blocks are frozen, and the stem and head remain trainable. Inference uses greedy CTC decoding: the most probable label is selected at each valid frame, repeats are merged, and blanks are removed. No external language model is used.

G. Experimental Configurations

Table I lists the systems. All systems share the same frontend, partitions, vocabulary, CTC head, and decoding procedure. Comparing them isolates three factors: the value of upstream Audio-JEPA weights (A versus B), the value of in-domain Thai JEPA pretraining (B versus C), and whether Thai JEPA pretraining alone, without AudioSet weights, is sufficient (A versus D). A wav2vec 2.0 encoder fine-tuned

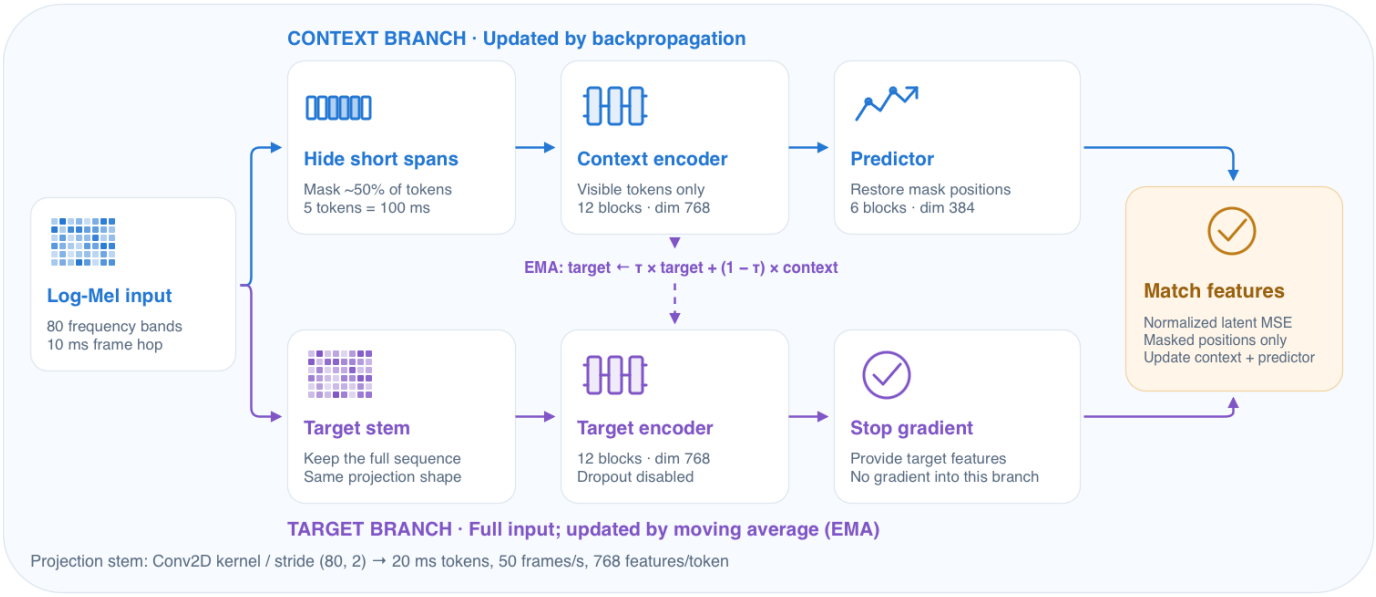


Fig. 2. Inside JEPA pretraining. The context branch sees only visible tokens and predicts latent features at the hidden positions. The target branch encodes the complete sequence and provides stop-gradient targets. Gradients update the context stem, context encoder, and predictor. The target branch is updated only by an exponential moving average (EMA) of the context weights.

TABLE I
EXPERIMENTAL CONFIGURATIONS. ALL SYSTEMS END WITH A NEWLY INITIALIZED CTC HEAD.

ID	Encoder initialization	Purpose
A	Random	Supervised-only control
B	Audio-JEPA (AudioSet)	Effect of upstream transfer
C	Audio-JEPA → Thai JEPA	Effect of in-domain Thai pretraining (proposed)
D	Random → Thai JEPA	Thai pretraining without AudioSet weights
Ref	wav2vec 2.0	Speech-specialized SSL reference

TABLE II
OPTIMIZATION SETTINGS FOR BOTH TRAINING STAGES.

Setting	Value
Stage 1: JEPA pretraining	
Optimizer	AdamW, learning rate 1×10^{-4}, weight decay 0.01
Batch / epochs	8 utterances per batch; 8 epochs
Masking	5-token (100 ms) spans, about 50% of valid tokens; 2 target sets
Predictor	6 blocks, width 384, 12 heads
EMA	τ: 0.996 → 1.0, cosine schedule
Stage 2: CTC fine-tuning	
Learning rates	Head 3×10^{-4}; stem 3×10^{-4} (1×10^{-4} after Thai JEPA); backbone 3×10^{-5} if pretrained, 3×10^{-4} if random
Batch / epochs	8 utterances, up to 15 epochs (1 h budget: 4 utterances, up to 25 epochs)
Early stopping	Patience of 5 validation evaluations (8 for the 1 h budget)
Regularization	Dropout 0.1 before the CTC head; weight decay 0.01
Decoding	Greedy CTC, no language model

with an identical CTC head serves as a speech-specialized self-supervised reference [4]. Its pretraining data and model size differ from those of our encoder, so it is a practical reference point rather than a controlled comparison of objectives.

To measure label efficiency, CTC fine-tuning is repeated with nested labeled subsets of 1, 10, and 50 hours and with the full training partition of approximately 700 hours. Each smaller subset is contained in every larger one. Self-supervised pretraining always uses the same unlabeled training audio, so only the transcription budget varies across conditions. Each configuration is trained with three random seeds (42, 43, and 44), and results are reported as mean ± standard deviation.

H. Training Details

Table II lists the optimization settings, taken from the experiment configurations in our codebase. Both stages use AdamW with a linear warmup over the first 10% of updates, followed by cosine decay to 1% of the peak learning rate, and gradient clipping at norm 1.0. Weight decay is not applied to biases or normalization parameters. The checkpoint with the lowest validation CER is selected, and training stops early when validation CER stops improving.

I. Evaluation Metrics

The primary metric is the character error rate (CER). Because Thai is written without spaces between words, word error rate (WER) requires word segmentation. We segment

both references and hypotheses with the PyThaiNLP `newmm` dictionary-based tokenizer [37], with the same version and dictionary for all systems, and then compute WER. For both units,

$$\mathrm{ER} = \frac{S + D + I}{N} \times 100\%, \tag{6}$$

where S, D, and I are the numbers of substitutions, deletions, and insertions, and N is the number of reference units. Counts are accumulated over the whole evaluation set rather than averaged per utterance. Model selection uses only the validation partition. The test partition is evaluated once, with the selected checkpoint of each run.

IV. Experimental Results and Discussion

This section will be completed after the experiments have been conducted.

V. Conclusion and Future Work

This section will be completed after the experiments have been conducted.

Acknowledgment

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

References

[1] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," in *Proceedings of Interspeech*, 2021, pp. 2426–2430.

[2] M. Shao, B. Mu, C. Wang, H. Li, Y. Yan, Z. Fu, and L. Xie, "Towards building speech large language models for multitask understanding in low-resource languages," *arXiv preprint arXiv:2509.14804*, 2025. [Online]. Available: https://arxiv.org/abs/2509.14804

[3] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[4] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 12 449–12 460.

[5] W.-N. Hsu *et al.*, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[6] Y. LeCun, "A path towards autonomous machine intelligence," OpenReview, 2022, version 0.9.2.

[7] M. Assran *et al.*, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of IEEE/CVF CVPR*, 2023, pp. 15 619–15 629.

[8] L. Tuncay, E. Labbé, E. Benetos, and T. Pellegrini, "Audio-JEPA: Joint-embedding predictive architecture for audio representation learning," in *Proceedings of IEEE International Conference on Multimedia and Expo (ICME)*, 2025. [Online]. Available: https://arxiv.org/abs/2507.02915

[9] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proceedings of ICML*, 2006, pp. 369–376.

[10] M. J. F. Gales and S. J. Young, "The application of hidden Markov models in speech recognition," *Foundations and Trends in Signal Processing*, vol. 1, no. 3, pp. 195–304, 2008.

[11] G. Hinton *et al.*, "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.

[12] H. Hermansky, "Perceptual linear predictive analysis of speech," *The Journal of the Acoustical Society of America*, vol. 87, no. 4, pp. 1738–1752, 1990.

[13] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[14] O. Abdel-Hamid, A. rahman Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 10, pp. 1533–1545, 2014.

[15] A. Graves, A. rahman Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proceedings of IEEE ICASSP*, 2013, pp. 6645–6649.

[16] W. Chan, N. Jaitly, Q. V. Le, and O. Vinyals, "Listen, attend and spell: A neural network for large vocabulary conversational speech recognition," in *Proceedings of IEEE ICASSP*, 2016, pp. 4960–4964.

[17] A. Gulati *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Proceedings of Interspeech*, 2020, pp. 5036–5040.

[18] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[19] Y.-A. Chung and J. Glass, "Generative pre-training for speech with autoregressive predictive coding," in *Proceedings of IEEE ICASSP*, 2020, pp. 3497–3501.

[20] S. Chen *et al.*, "WavLM: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.

[21] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, "data2vec: A general framework for self-supervised learning in speech, vision and language," in *Proceedings of ICML*, vol. 162, 2022, pp. 1298–1312.

[22] A. Mohamed, H. yi Lee, L. Borgholt, J. D. Havtorn, J. Edin, C. Igel, K. Kirchhoff, S.-W. Li, K. Livescu, L. Maaløe, T. N. Sainath, and S. Watanabe, "Self-supervised speech representation learning: A review," *IEEE Journal of Selected Topics in Signal Processing*, 2022.

[23] A. T. Liu, S. wen Yang, P.-H. Chi, P. chun Hsu, and H. yi Lee, "Mockingjay: Unsupervised speech representation learning with deep bidirectional transformer encoders," in *Proceedings of IEEE ICASSP*, 2020, pp. 6419–6423.

[24] S. Ling, Y. Liu, J. Salazar, and K. Kirchhoff, "Deep contextualized acoustic representations for semi-supervised speech recognition," in *Proceedings of IEEE ICASSP*, 2020, pp. 6429–6433.

[25] A. T. Liu, S.-W. Li, and H. yi Lee, "TERA: Self-supervised learning of transformer encoder representation for speech," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 2351–2366, 2021.

[26] Z. Fei, M. Fan, and J. Huang, "A-JEPA: Joint-embedding predictive architecture can listen," *arXiv preprint arXiv:2311.15830*, 2023, revised January 2024. [Online]. Available: https://arxiv.org/abs/2311.15830

[27] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, "Investigating design choices in joint-embedding predictive architectures for general audio representation learning," *arXiv preprint arXiv:2405.08679*, 2024. [Online]. Available: https://arxiv.org/abs/2405.08679

[28] G. Ioannides, A. Kieback, J. Goldfeder, L. Pang, A. Chadha, A. Elkins, Y. LeCun, and R. Shwartz-Ziv, "Soft clustering anchors for self-supervised speech representation learning in joint embedding prediction architectures," *arXiv preprint arXiv:2602.09040*, 2026. [Online]. Available: https://arxiv.org/abs/2602.09040

[29] ——, "S-JEPA: Soft clustering anchors for self-supervised speech representation learning," *arXiv preprint arXiv:2606.19398*, 2026. [Online]. Available: https://arxiv.org/abs/2606.19398

[30] A. Babu, C. Wang, A. Tjandra, K. Lakhotia, Q. Xu, N. Goyal, K. Singh, P. von Platen, Y. Saraf, J. Pino, A. Baevski, A. Conneau, and M. Auli, "XLS-R: Self-supervised cross-lingual speech representation learning at scale," in *Proceedings of Interspeech*, 2022, pp. 2278–2282.

[31] V. Pratap, A. Tjandra, B. Shi, P. Tomasello, A. Babu, S. Kundu, A. Elkahky, Z. Ni, A. Vyas, M. Fazel-Zarandi, A. Baevski, Y. Adi, X. Zhang, W.-N. Hsu, A. Conneau, and M. Auli, "Scaling speech technology to 1,000+ languages," *arXiv preprint arXiv:2305.13516*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13516

[32] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of ICML*, vol. 202, 2023, pp. 28 492–28 518.

[33] W. Sirichotedumrong, A. Na-Thalang, P. Manakul, P. Taveekitworachai, S. Sripaisarnmongkol, and K. Pipatanakul, "Typhoon ASR real-time:

FastConformer-Transducer for Thai automatic speech recognition," *arXiv preprint arXiv:2601.13044*, 2026. [Online]. Available: https://arxiv.org/abs/2601.13044

[34] CMKL University, "Porjai Thai voice dataset (central)," Hugging Face dataset https://huggingface.co/datasets/CMKL/Porjai-Thai-voice-dataset-central, 2024, cC BY-SA 4.0.

[35] D. Berrebbi, J. Shi, B. Yan, O. Lopez-Francisco, J. D. Amith, and S. Watanabe, "Combining spectral and self-supervised features for low resource speech recognition and translation," *arXiv preprint arXiv:2204.02470*, 2022. [Online]. Available: https://arxiv.org/abs/2204.02470

[36] L. Tuncay, "Audio-JEPA pretrained checkpoint," https://huggingface.co/ltuncay/Audio-JEPA, 2025.

[37] W. Phatthiyaphaibun, K. Chaovavanich, C. Polpanumas, A. Suriyawongkul, L. Lowphansirikul, P. Chormai, P. Limkonchotiwat, T. Suntorntip, and C. Udomcharoenchaikit, "PyThaiNLP: Thai natural language processing in Python," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023, pp. 25–36.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.